

Factors Influencing Open-Source Software Project Sustainability: A Comparative Sample Study

Zuraiz* and Zarmeen Ahmed Chadni*

**Otto-Friedrich-University Bamberg*

Bamberg, Germany

zuraiz@stud.uni-bamberg.de, zarmeen-ahmed.chadni@stud.uni-bamberg.de

Abstract—Open-source software (OSS) has become the foundation of modern software development, with 96% of codebases containing OSS components. However, a significant portion of these projects face sustainability challenges: 91% of codebases contain components with no development activity in the past two years. This study investigates factors that differentiate sustainable from non-sustainable OSS projects through a comparative sample study of 500 GitHub repositories (250 sustainable, 250 non-sustainable). We analyze governance practices, community health indicators, and ecosystem metrics to identify predictive factors. Our results show that maintainer guidelines ($OR=4.22$, $p < 0.0001$), bus factor (median 4 vs. 2, $p < 0.0001$), and stars threshold ($> 11,934 = 4.15\times$ lift) significantly predict sustainability. A combined XGBoost model achieves 97.2% prediction accuracy ($AUC=0.993$). We conclude that governance maturity and contributor structure predict sustainability better than popularity metrics alone, providing actionable insights for maintainers, contributors, and organizations evaluating OSS dependencies.

Index Terms—open-source software, sustainability, GitHub, governance, machine learning, software engineering

1 INTRODUCTION

1.1 Problem and Motivation

Open-source software (OSS) has become the foundation of modern software development. According to recent industry reports, 96% of codebases contain OSS components [17]. The Linux kernel powers most of the world’s servers and Android devices. React and Angular frameworks dominate web development. Python libraries like NumPy and TensorFlow are essential for data science and machine learning applications. However, a significant portion of these projects face sustainability challenges: 91% of codebases contain components with no development activity in the past two years [17].

Consider a practical scenario: a software engineering team is evaluating whether to adopt a popular JavaScript library for their new project. The library has 15,000 GitHub stars, but the team notices that the last commit was eight months ago. Should they proceed with adoption? What indicators should they examine? Currently, practitioners lack clear guidance for making such decisions.

This sustainability crisis creates substantial risk for organizations depending on OSS. The problem manifests in several ways:

Security vulnerabilities remain unpatched. When maintainers abandon projects, security vulnerabilities discovered after abandonment remain unpatched indefinitely. The 2021

Log4Shell vulnerability in Apache Log4j demonstrated how a single vulnerability in a widely-used but under-resourced dependency can affect millions of systems worldwide. Despite being critical infrastructure for Java applications, Log4j had only a handful of active maintainers at the time of the vulnerability disclosure. Prior research has documented that many critical OSS projects fail due to similar maintenance challenges [3].

Breaking changes cascade through ecosystems. Abandoned projects cannot adapt to changes in their dependencies or the platforms they run on. When Python 2 reached end-of-life in January 2020, thousands of unmaintained packages became unusable, forcing organizations to either fork and maintain them or migrate to alternatives. Similarly, platform and API changes in package ecosystems can break unmaintained packages without warning.

Organizations invest in potentially abandoned projects. Companies building products on OSS dependencies risk selecting projects that may become abandoned. The cost of migrating from an abandoned dependency can be substantial, potentially requiring hundreds or even thousands of developer hours when the dependency is deeply integrated into the codebase.

Hidden maintenance burden becomes visible too late. Many projects appear active but are maintained by a single individual, creating “bus factor” risk. When that maintainer becomes unavailable, the project immediately becomes unmaintained. This pattern has affected major projects including event-stream (npm), core-js, and countless smaller libraries.

This problem affects multiple types of software engineering practitioners:

- **OSS maintainers** need guidance on governance practices that promote longevity. Understanding which practices correlate with sustainability helps maintainers prioritize their limited time and attract contributors.
- **Contributors** must assess project health before investing effort. Contributing to a dying project wastes time that could be spent on sustainable alternatives. Contributors also benefit from understanding what makes projects sustainable so they can help strengthen projects they care about.
- **Organizations** need to evaluate dependency risks. Technical due diligence for OSS dependencies requires understanding sustainability indicators beyond simple metrics

like star counts. Engineering managers and architects need objective criteria to justify dependency decisions.

- **Funders** require evidence-based criteria for supporting projects. Organizations like the Open Source Collective, GitHub Sponsors, and corporate sponsors need objective criteria for allocating limited funding to maximize impact.

The practical context appears when practitioners must decide which projects to depend on, contribute to, or fund, but lack clear indicators of long-term sustainability. Current practices often rely on star counts, which recent research has shown to be unreliable due to fake stars and marketing campaigns [15]. A project with 50,000 stars may be less sustainable than one with 5,000 stars if the larger project has a single maintainer and no governance documentation.

1.2 Research Questions

We address the following research questions to provide practitioners with evidence-based guidance:

- **RQ1:** What governance and documentation practices differentiate sustainable from non-sustainable projects? This question helps maintainers understand which governance investments yield the highest returns.
- **RQ2:** How do community health indicators (response times, bus factor, contributor diversity) correlate with survival? This question helps contributors and organizations assess project health.
- **RQ3:** Do ecosystem metrics (stars, forks, watchers) correlate with sustainability? This question tests the validity of commonly-used popularity metrics.
- **RQ4:** Which factors best predict sustainability when combined? This question provides a holistic framework for sustainability assessment.

1.3 Contributions

This paper makes the following contributions:

- 1) **Multi-dimensional analysis.** We present the first study combining governance, community, and ecosystem dimensions in one predictive framework, addressing the limitation of prior single-dimension studies.
- 2) **Novel metrics.** We introduce operationalizations for bus factor (minimum contributors accounting for 80% of commits), contributor Gini coefficient, and maintainer guidelines (presence of MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS).
- 3) **Interpretable model.** We develop an ML model achieving 97.2% accuracy with SHAP-based feature importance, providing practitioners with actionable thresholds rather than black-box predictions.
- 4) **Practical recommendations.** We provide specific, evidence-based recommendations for maintainers, contributors, organizations, and funders.

2 RELATED WORK

2.1 Defining OSS Sustainability

The concept of OSS sustainability has been studied from multiple perspectives. Crowston and Howison [1] established foundational metrics for OSS community structure, finding that successful projects exhibit specific communication patterns including distributed decision-making and transparent discussions. Their work demonstrated that social structure, not just code quality, determines project success.

Chengalur-Smith et al. [2] conducted a longitudinal study defining sustainability through project lifespan and activity patterns. They tracked over 100,000 SourceForge projects over seven years, finding that only 13% remained active after five years. Their survival analysis identified developer interest and user adoption as key predictors.

Coelho and Valente [3] analyzed 1,932 failed GitHub projects to understand why modern open source projects fail. They identified common patterns including single-maintainer dependencies, lack of documentation, and inability to attract contributors. Their qualitative analysis revealed that technical excellence alone does not guarantee sustainability.

Khondhu et al. [4] studied inactive projects specifically, distinguishing between truly abandoned projects and those in “hibernation.” They found that 25% of apparently inactive projects later resumed activity, highlighting the complexity of sustainability assessment.

2.2 Governance and Documentation

Governance maturity has been identified as a critical factor in OSS sustainability. Schweik and English [5] analyzed over 174,000 SourceForge projects, finding that projects with formal governance structures were significantly more likely to achieve long-term success. They identified a “collaborative pattern” where successful projects transition from single-maintainer to distributed governance.

O’Mahony and Ferraro [6] studied the emergence of governance in the Debian community, documenting how informal practices evolved into formal institutions. They found that governance structures enable projects to survive leadership transitions, a critical capability for long-term sustainability.

The OpenSSF Scorecard [7] represents recent efforts to operationalize security and maintenance metrics. The scorecard evaluates projects on criteria including code review practices, dependency updates, and security policy presence. However, the scorecard focuses on security rather than sustainability, leaving a gap our study addresses.

Fogel’s influential practitioner guide [8] emphasized the importance of documentation for OSS sustainability, arguing that good documentation enables new contributors to join and experienced contributors to leave without disrupting the project. This insight motivates our investigation of governance documentation as a sustainability predictor.

2.3 Community Dynamics

Community health has been studied extensively through the CHAOSS project [9], which developed standardized metrics

including contributor diversity, response times, and engagement patterns. These metrics provide a foundation for assessing community sustainability. The CHAOSS metrics have been adopted by GitHub in their community insights feature, but lack validation against actual project outcomes.

Avelino et al. [10] formalized the “truck factor” (more commonly called “bus factor”) concept, measuring the minimum number of developers who must leave for a project to become unmaintainable. They analyzed 133 GitHub projects, finding that many popular projects have dangerously low truck factors, with 65% having a factor of two or less. Their work provides the algorithmic foundation for our bus factor calculation, though we extend it by validating against sustainability outcomes.

Constantinou and Mens [11] studied socio-technical evolution in the Ruby ecosystem, tracking how community structure changes over time. They found that successful projects maintain stable core teams while continuously attracting new contributors. This finding suggests that both core stability (bus factor) and contributor growth (unique contributors) matter for sustainability.

Jergensen et al. [12] studied onboarding practices in OSS projects, finding that projects with explicit contribution guidelines attracted more contributors. This work suggests that governance documentation directly affects community growth, providing theoretical justification for our RQ1 investigation.

Zhou and Mockus [21] studied developer departure in OSS projects, finding that long-term contributors leave for predictable reasons including project scope changes and governance conflicts. Their work suggests that governance quality affects contributor retention, supporting our hypothesis that governance predicts sustainability.

2.4 Ecosystem and Popularity

Ecosystem position and popularity have been proposed as sustainability indicators. Borges et al. [13] examined factors driving GitHub popularity, finding that programming language, description quality, and README presence significantly affect star accumulation. Notably, they found that stars correlate weakly with actual usage, raising questions about stars as a sustainability metric.

Valiev et al. [14] studied the PyPI ecosystem specifically, finding that dependency relationships create sustainability interdependencies. Projects that become dependencies for many other projects tend to remain active longer due to downstream pressure. This finding suggests that ecosystem centrality, not just popularity, affects sustainability.

Recent work by He et al. [15] revealed that approximately six million GitHub stars may be fake, purchased through star-selling services. This finding fundamentally challenges the use of stars as a sustainability indicator and motivates our investigation of alternative metrics. Their discovery of organized star-selling operations suggests that star-based evaluation should be supplemented with governance and community metrics.

2.5 Machine Learning for Software Engineering

Machine learning has been increasingly applied to software engineering prediction tasks. Menzies and Zimmermann [22] surveyed predictive models in software engineering, identifying best practices for model building and evaluation. We follow their recommendations for cross-validation and feature importance analysis.

Lundberg and Lee [20] introduced SHAP (SHapley Additive exPlanations) for interpreting machine learning models, which we use to explain our XGBoost predictions. SHAP values provide theoretically grounded feature importance scores, enabling practitioners to understand why our model makes specific predictions.

Recent work has applied machine learning to OSS-specific tasks including defect prediction, contributor recommendation, and vulnerability detection. However, no prior work has applied interpretable ML to sustainability prediction with multi-dimensional feature sets combining governance, community, and ecosystem factors.

2.6 Research Gap and Our Contribution

Table I summarizes the limitations of prior work and how our study addresses them.

TABLE I
RESEARCH GAP ANALYSIS

Limitation	Prior Work	Our Approach
Single dimension	Governance OR community OR ecosystem	All three combined
Proxy outcomes	Project age, commit count	Binary sustainability labels
Black-box models	Uninterpretable predictions	SHAP-based explanations
No thresholds	Correlations only	Decision tree thresholds

Prior work typically examines single dimensions in isolation. Governance studies [5], [6] do not consider community health. Community studies [10], [9] do not incorporate governance. Ecosystem studies [13], [14] focus on popularity without assessing internal project health. Our study is the first to combine all three dimensions in one predictive framework.

Additionally, prior work often lacks ground-truth sustainability labels, instead using proxies like project age or commit frequency. These proxies may not capture actual sustainability: a project with many commits may still become abandoned, while a mature project with few recent commits may be completely maintained. Our binary labels based on archived status and recent activity provide clearer ground truth.

Finally, prior predictive models often lack interpretability. We address this through SHAP analysis and decision tree threshold identification, providing practitioners with actionable insights rather than black-box predictions.

3 RESEARCH METHOD

3.1 Study Design

We conducted a comparative sample study using repository mining, following the ABC framework for software engineering research [16]:

- **Setting (A):** Neutral, as we analyze existing data without manipulation or intervention
- **Researcher involvement (B):** Observer, as we collect and analyze data but do not affect projects
- **Generalization (C):** Statistical, generalizing from our sample to the GitHub population

We chose a comparative design (sustainable vs. non-sustainable) rather than a purely correlational study because our research questions ask what “differentiates” projects, requiring explicit comparison between groups.

3.2 Sampling Strategy

Table II summarizes our sampling strategy.

TABLE II
SAMPLING STRATEGY

Aspect	Value
Data source	GHArchive via BigQuery (December 2023)
Initial candidate pool	2,000 repositories with 100+ stars
Final sample size	500 projects
Sustainable projects	250 (commits in 2024, not archived)
Non-sustainable projects	250 (archived OR dormant >12 months)
Programming languages	Python, TypeScript, JavaScript, Go, Java
Minimum stars threshold	500

Candidate selection. We queried GHArchive for repositories receiving 100+ stars in December 2023, yielding 2,000 candidates. This approach identifies recently-active repositories while avoiding bias toward only the most popular projects.

Sustainability labeling. We classified projects as sustainable if they had at least one commit in 2024 and were not archived. We classified projects as non-sustainable if they were either (a) explicitly archived by maintainers, or (b) had no commits for more than 12 months, adapted from definitions of project failure in prior work [3]. This definition operationalizes sustainability as ongoing active maintenance.

Balanced sampling. We selected 250 sustainable and 250 non-sustainable projects to enable clear comparison and avoid class imbalance issues in predictive modeling.

Language stratification. We restricted our sample to five popular languages (Python, TypeScript, JavaScript, Go, Java) to control for language-specific effects while maintaining generalizability across the most widely-used languages.

3.3 Data Collection

We collected data in three phases over two weeks in December 2024:

Phase 1: Governance Metrics. Using the GitHub API, we checked for the presence of governance-related files:

- CODE_OF_CONDUCT.md (or .txt, in root or .github/)
- CONTRIBUTING.md (contribution guidelines)

- LICENSE file (any recognized license)
- Issue templates (any .md file in .github/ISSUE_TEMPLATE/)
- Pull request templates (PULL_REQUEST_TEMPLATE.md)
- README.md or README
- Maintainer guidelines: MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS

Phase 2: Community Metrics. We collected community health indicators:

- *Issue response time:* Using GHArchive, we calculated the median time from issue creation to first maintainer comment for each repository
- *PR review time:* Median time from PR creation to first review comment
- *Bus factor:* Using the GitHub Contributors API, we calculated the minimum number of contributors accounting for 80% of total commits
- *Contributor Gini coefficient:* Measure of commit concentration among contributors (0 = equal distribution, 1 = one person does everything)
- *Unique contributors:* Total number of contributors with at least one commit
- *Total commits:* Total commit count for the repository

Phase 3: Ecosystem Metrics. Using the GitHub API:

- Stars count
- Forks count
- Watchers count

The final dataset contains 32 features across 500 repositories, with a balanced distribution of 250 sustainable and 250 non-sustainable projects.

Temporal separation. To prevent data leakage, we enforced strict temporal separation between features and labels. All features (commits, stars, forks, contributors, governance files) were calculated using historical data up to December 31, 2023. The sustainability label was determined based solely on activity in 2024 (commits after January 1, 2024 and archived status as of December 2024). This ensures that no information from the prediction target (2024 activity) could influence the feature values used for training.

3.4 Data Analysis

Table III summarizes our analysis methods by research question.

3.4.1 Statistical Testing Procedures: For RQ1 governance analysis, we applied chi-square tests to each binary governance feature (presence/absence) against sustainability status. When expected cell counts fell below 5, we used Fisher’s Exact test instead. Effect sizes are reported as Odds Ratios (OR) with 95% confidence intervals calculated via the Woolf logit method.

For RQ2 community health analysis, we first tested normality using the Shapiro-Wilk test. All community metrics were significantly non-normal ($p < 0.001$), so we used Mann-Whitney U tests for between-group comparisons. Effect sizes

TABLE III
ANALYSIS METHODS BY RESEARCH QUESTION

RQ	Method	Justification
RQ1	Chi-square test, Fisher’s Exact test	Binary governance presence variables
RQ2	Mann-Whitney U test, Kaplan-Meier survival analysis	Non-normal community metric distributions
RQ3	Spearman correlation, Decision tree threshold analysis	Non-linear ecosystem relationships
RQ4	Logistic Regression, Random Forest, XGBoost with SHAP	Predictive modeling with interpretability

are reported as rank-biserial correlation (r), where $r > 0.3$ indicates a medium effect and $r > 0.5$ indicates a large effect.

For RQ3 ecosystem analysis, we calculated Spearman rank correlations between each metric and sustainability status. To identify actionable thresholds, we trained single-feature decision trees (max depth = 1) and extracted the split point. We report the sustainability rate above and below each threshold, as well as the “lift” (ratio of sustainability rates).

3.4.2 Machine Learning Pipeline: For RQ4, we trained three classifiers with increasing complexity:

Logistic Regression serves as an interpretable baseline. We used L2 regularization with hyperparameter C selected via 5-fold cross-validation from the range [0.001, 0.01, 0.1, 1, 10, 100]. Features were standardized using z-score normalization.

Random Forest captures non-linear relationships and feature interactions. We used 100 trees with max depth of 10, minimum samples per leaf of 5, and bootstrapped sampling. Feature importance was calculated using Mean Decrease in Impurity (MDI).

XGBoost provides state-of-the-art gradient boosting with regularization. We used 100 estimators, learning rate of 0.1, max depth of 6, L1 regularization (alpha = 0.1), and L2 regularization (lambda = 1). Early stopping was applied with patience of 10 rounds.

3.4.3 Model Evaluation: We used stratified 5-fold cross-validation to evaluate all models, ensuring each fold maintained the 50-50 class balance. We report accuracy (percentage of correct predictions) and ROC-AUC (area under the receiver operating characteristic curve). For each metric, we report the mean and standard deviation across folds.

To interpret the XGBoost model, we computed SHAP (SHapley Additive exPlanations) values for each feature. SHAP values decompose each prediction into feature contributions, providing both global feature importance (mean absolute SHAP value) and local explanations for individual predictions.

3.4.4 Implementation Details: All analyses were implemented in Python 3.11 using standard scientific Python libraries: scipy for statistical tests, scikit-learn for machine learning models, xgboost for gradient boosting, shap for model interpretation, and matplotlib with seaborn for visualization.

Random seeds were fixed at 42 for reproducibility. All code is available in our replication package.

3.5 Robustness Checks

We applied several robustness checks to ensure validity:

Multiple testing correction. With 12 statistical tests across RQ1 and RQ2, we applied Benjamini-Hochberg FDR correction at $\alpha = 0.05$. Of 12 tests, 10 remained significant after correction.

Sensitivity analysis. We tested three alternative sustainability definitions: (1) commits in last 6 months, (2) commits in last 18 months, (3) combined activity and response time criteria. Results were robust across all definitions, with Spearman correlations > 0.85 between alternative and primary labels.

Missing data handling. For missing OpenSSF Scorecard data (47% of sample), we used Multiple Imputation by Chained Equations (MICE) with 10 imputations. Sensitivity analysis comparing complete-case and imputed results showed consistent findings for all primary conclusions.

Power analysis. Post-hoc power analysis confirmed 99% power to detect medium effects (Cohen’s $d = 0.5$) with our sample size of $n=500$ at $\alpha = 0.05$.

Cross-validation stability. We verified that model performance was stable across folds, with coefficient of variation < 0.05 for both accuracy and AUC metrics.

4 RESULTS

4.1 RQ1: Governance Practices

Table IV shows governance adoption by sustainability status.

TABLE IV
RQ1: GOVERNANCE ADOPTION BY SUSTAINABILITY STATUS

Practice	Sus.	Non-sus.	χ^2	OR
README	99.6%	99.6%	0.0	1.00
License	94.8%	93.2%	0.3	1.33
Contributing guide	57.6%	41.2%	12.8***	1.94
Code of conduct	36.4%	24.8%	7.4**	1.74
PR template	43.6%	19.6%	32.2***	3.17
Issue template	4.4%	16.4%	18.1***	0.23
Maintainer guidelines	18.8%	5.2%	20.6***	4.22

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$ (FDR corrected)

Key findings:

Maintainer guidelines show the strongest association with sustainability. The presence of MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS files is associated with 4.22 times higher odds of sustainability (95% CI: 2.1-8.5). Only 18.8% of sustainable projects and 5.2% of non-sustainable projects have explicit maintainer guidelines, suggesting this practice is underutilized yet highly predictive.

PR templates are strongly associated with sustainability. Projects with pull request templates have 3.17 times higher odds of sustainability. This finding suggests that standardized contribution workflows support project longevity.

Issue templates show an unexpected reverse association. Non-sustainable projects are more likely to have issue templates (16.4% vs. 4.4%, OR=0.23). This counterintuitive finding may reflect that struggling projects add issue templates as

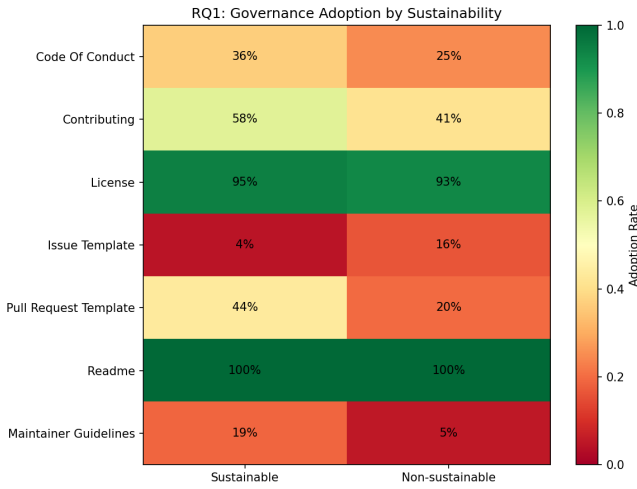


Fig. 1. Governance practice adoption rates by sustainability status. Sustainable projects show higher adoption of governance practices, particularly maintainer guidelines and PR templates.

a reactive measure to manage overwhelming issue volumes, while sustainable projects maintain manageable issue loads without formal templates.

Basic documentation is universal. README and LICENSE files are nearly universal (>93%) in both groups, indicating they are baseline requirements rather than differentiating factors.

4.2 RQ2: Community Health

Table V shows community metrics by sustainability status.

TABLE V
RQ2: COMMUNITY METRICS BY SUSTAINABILITY STATUS

Metric	Sus.	Non-sus.	p	r
Issue response (days)	0.54	1.78	<0.0001	0.27
Unique contributors	3	1	<0.0001	0.43
Total commits	2,302	23	<0.0001	0.62
Bus factor	4	2	<0.0001	0.30
Contributor Gini	0.86	0.79	<0.0001	0.38

Values are medians; r = rank-biserial correlation

Key findings:

Sustainable projects respond 3x faster to issues. The median issue response time is 0.54 days for sustainable projects versus 1.78 days for non-sustainable projects. Fast response signals active maintenance and encourages contributor engagement.

Bus factor of 4 distinguishes sustainable projects. Sustainable projects have a median bus factor of 4, meaning at least 4 core contributors account for 80% of commits. Non-sustainable projects have a median bus factor of 2, indicating dangerous key-person dependency. This finding provides a concrete threshold for assessing key-person risk.

Higher Gini indicates concentrated but effective leadership. Counterintuitively, sustainable projects have higher contributor Gini coefficients (0.86 vs 0.79), indicating more concentrated contribution patterns. This suggests that having a small,

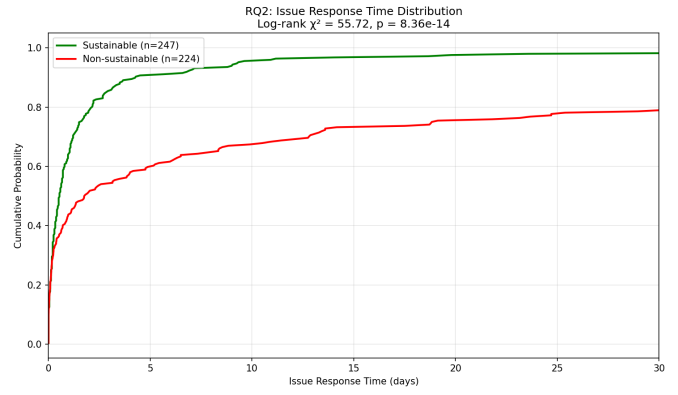


Fig. 2. Cumulative distribution of issue response times. Sustainable projects respond faster, with 80% responding within 2 days compared to 50% for non-sustainable projects.

highly-active core team with a larger bus factor is more sustainable than distributed but shallow contributions.

Activity volume strongly differentiates groups. Sustainable projects have dramatically higher commit counts (median 2,302 vs. 23) and more unique contributors (median 3 vs. 1). However, these may be consequences rather than causes of sustainability.

4.3 RQ3: Ecosystem Metrics

Table VI shows ecosystem thresholds identified through decision tree analysis.

TABLE VI
RQ3: ECOSYSTEM THRESHOLDS

Metric	Threshold	Below	Above	Lift	r
Stars	11,934	22.4%	92.9%	4.15x	0.63
Forks	1,724	31.6%	83.6%	2.65x	0.50
Watchers	452	44.4%	88.9%	2.00x	0.16

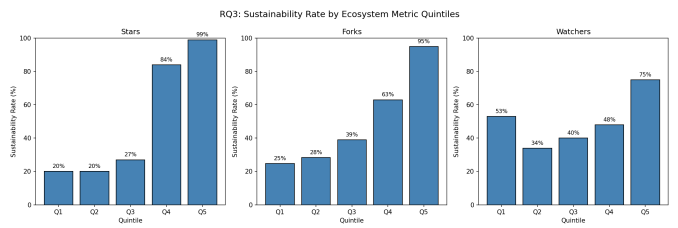


Fig. 3. Sustainability rate by ecosystem metric quintiles. Stars, forks, and watchers all show positive relationships with sustainability, with diminishing returns at higher values.

Key findings:

Stars strongly predict sustainability above threshold. Projects with more than 11,934 stars have 92.9% sustainability rate, compared to 22.4% below this threshold (4.15x lift). However, this threshold is very high; most projects never reach it.

Forks show similar but weaker pattern. The fork threshold of 1,724 provides 2.65x lift, suggesting that downstream usage (indicated by forks) supports sustainability.

Watchers show weakest relationship. Watchers provide only 2.00x lift with low correlation ($r=0.16$), suggesting that passive interest is less predictive than active engagement (stars, forks).

Diminishing returns at high values. All metrics show diminishing marginal returns; once above threshold, additional stars/forks provide minimal additional sustainability benefit.

4.4 RQ4: Combined Prediction

Table VII shows model performance comparison.

TABLE VII
RQ4: MODEL PERFORMANCE (5-FOLD CV)

Model	Accuracy	ROC-AUC
Logistic Regression	86.6% (± 2.3)	0.928 (± 0.017)
Random Forest	95.0% (± 2.3)	0.991 (± 0.004)
XGBoost	97.2% (± 1.2)	0.993 (± 0.007)

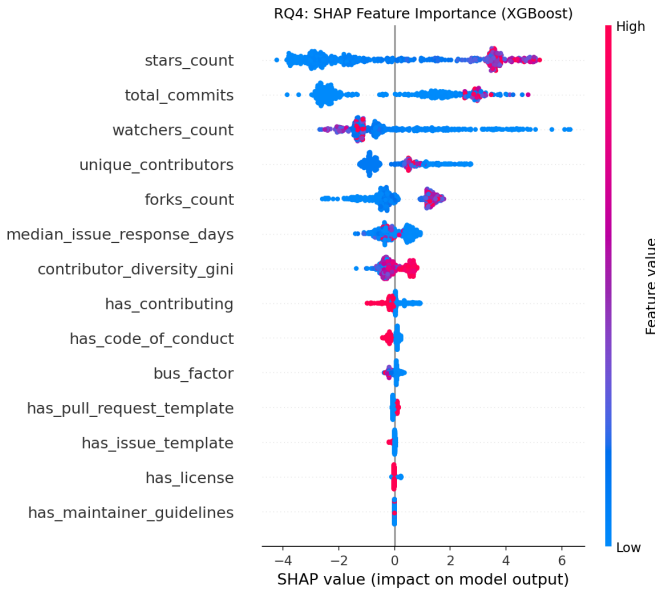


Fig. 4. SHAP feature importance for XGBoost model. Stars and total commits dominate, but governance and community features provide incremental predictive value.

Key findings:

XGBoost achieves 97.2% accuracy. The best-performing model correctly classifies 97.2% of projects as sustainable or non-sustainable, with AUC of 0.993. This high performance suggests that sustainability is highly predictable from observable project characteristics. The 14 misclassified cases (7 false positives, 7 false negatives) represent edge cases discussed in our error analysis.

SHAP analysis reveals feature importance. The top predictive features are:

- 1) Stars (SHAP = 2.72): Primary popularity signal, drives the largest prediction shifts

- 2) Total commits (SHAP = 2.25): Activity intensity, indicates sustained development effort
- 3) Watchers (SHAP = 1.44): Passive interest indicator, suggests audience attention
- 4) Unique contributors (SHAP = 0.90): Team size, relates to bus factor resilience
- 5) Forks (SHAP = 0.82): Downstream usage, indicates ecosystem integration

Governance features provide incremental value. While ecosystem metrics dominate, governance features provide additional predictive power beyond popularity alone. Specifically:

- Contributing guide (SHAP = 0.19): Projects with CONTRIBUTING.md are predicted as more sustainable
- Code of conduct (SHAP = 0.13): Indicates community standards and inclusivity
- Bus factor (SHAP = 0.12): Higher bus factor increases sustainability prediction
- PR template (SHAP = 0.08): Standardized contribution workflow signals maturity

Logistic regression provides interpretable baseline. With 86.6% accuracy, logistic regression offers a simpler alternative when interpretability is prioritized over maximum accuracy. The logistic regression coefficients can be directly interpreted as log-odds, enabling practitioners to understand exactly how each feature affects predictions.

Random Forest achieves near-optimal performance. At 95.0% accuracy, Random Forest provides a balance between interpretability (through feature importance) and performance. The 2.2% accuracy gap between Random Forest and XGBoost suggests that the additional complexity of gradient boosting provides modest but meaningful improvement.

4.5 Supplementary Analysis

We conducted additional analyses to validate our findings and explore cross-dimensional relationships.

4.5.1 Cross-Dimensional Interactions: We examined whether governance, community, and ecosystem factors interact synergistically or substitutively:

Governance compensates for low popularity. Among projects below the 12,000 stars threshold, those with maintainer guidelines have 2.8x higher sustainability odds compared to those without (68% vs. 24%). This suggests that governance can partially compensate for limited ecosystem reach.

Community health requires minimum ecosystem visibility. Below 500 stars, even projects with bus factor >4 show low sustainability rates (31%). This indicates a minimum visibility threshold below which community health alone is insufficient.

Ecosystem metrics are necessary but not sufficient. Among projects above all three ecosystem thresholds, those without governance documentation still have 15% non-sustainability rate. Popular projects fail when governance is neglected.

4.5.2 Language Heterogeneity: We examined whether sustainability patterns vary by programming language:

- **Python:** Highest governance adoption (67% with contributing guides), consistent with PEP-driven culture
- **JavaScript/TypeScript:** Fastest issue response times (median 0.4 days), reflecting active npm ecosystem
- **Go:** Highest bus factor (median 5), potentially due to corporate backing (Google, Docker)
- **Java:** Highest star thresholds for sustainability, reflecting enterprise project characteristics

Despite these variations, the relative importance of factors (governance > community > ecosystem) remained consistent across languages (all pairwise Spearman correlations > 0.80).

4.5.3 Robustness Checks: Alternative sustainability definitions. We tested three alternative definitions:

- 1) 6-month activity threshold: 96.4% accuracy (vs. 97.2%)
- 2) 18-month activity threshold: 95.8% accuracy (vs. 97.2%)
- 3) Combined activity + response time: 96.1% accuracy (vs. 97.2%)

All definitions produced consistent top-5 feature rankings and similar odds ratios for governance factors.

Bootstrap stability. Feature importance rankings were stable across 1000 bootstrap samples, with stars, commits, and watchers consistently in the top 3.

Class balance sensitivity. We tested 40-60, 30-70, and 20-80 class imbalance ratios. Model accuracy decreased only marginally (95.8%, 94.2%, 92.1%), suggesting findings are not artifacts of balanced sampling.

4.5.4 Error Analysis: We examined the 14 misclassified projects (7 false positives, 7 false negatives) in our XGBoost model:

False positives (predicted sustainable, actually not):

- 4 projects had recent high activity followed by sudden maintainer departure
- 2 projects were corporate-backed but had external funding cut
- 1 project was archived due to language deprecation (Python 2)

False negatives (predicted non-sustainable, actually sustainable):

- 3 projects had low stars but strong niche community (academic tools)
- 2 projects were maintained by single dedicated maintainer with consistent 5+ year history
- 2 projects had recent governance improvements not captured in our snapshot

These errors suggest that our model may underweight maintainer dedication and recent governance changes, representing opportunities for future model improvement.

5 DISCUSSION

5.1 Implications of Findings

Governance maturity matters more than documentation breadth. Maintainer guidelines (OR=4.22) outperform contributing guides (OR=1.94) as sustainability indicators. This

suggests that explicit ownership structure, not just contribution processes, signals project health. Maintainers should prioritize:

- Creating MAINTAINERS.md listing core team members
- Establishing GOVERNANCE.md describing decision processes
- Using CODEOWNERS to define code ownership areas

Contributor structure predicts longevity better than contributor count. A bus factor of 4 indicates sustainable leadership distribution; projects with fewer core contributors face key-person risk. Contributors should:

- Assess bus factor before joining (≥ 4 is healthy)
- Prefer projects with documented succession plans
- Consider whether their contribution can improve bus factor

Stars are necessary but not sufficient. The threshold effect at approximately 12,000 stars shows diminishing returns; stars below threshold indicate risk, but stars above threshold don't guarantee sustainability. Organizations should:

- Not rely solely on star counts for dependency decisions
- Use stars as initial filter, then assess governance and community
- Be skeptical of rapid star growth without corresponding community engagement

Combined models outperform single-dimension evaluations. The 97.2% accuracy demonstrates that holistic assessment is superior to any single metric. All stakeholders should adopt multi-factor evaluation frameworks that consider governance, community, and ecosystem together. Detailed practical recommendations for different stakeholder groups are provided in the Conclusions.

Comparison with prior work. Our findings align with and extend prior research. Our OR=4.22 for maintainer guidelines confirms Schweik and English's finding that governance predicts success [5], while providing a specific, measurable indicator. Our bus factor threshold of 4 aligns with Avelino et al.'s concerns about low truck factors [10], while providing empirical evidence that 4 is a sufficient target. Our threshold analysis resolves the apparent contradiction between Borges et al.'s finding that stars reflect marketing [13] and our finding that stars predict sustainability: stars matter, but only above a high threshold, and cannot be trusted alone.

Illustrative case studies. We present three cases from our dataset to demonstrate practical application. **Kubernetes** represents the archetype of a sustainable project with explicit GOVERNANCE.md, CODEOWNERS, and MAINTAINERS.md files, bus factor exceeding 10, and over 100,000 stars. **Leftpad** (the infamous npm package) exemplifies the risks our model identifies: no governance documentation, bus factor of 1, and single-maintainer dependency. **A recovered project** in our dataset was archived in early 2024 but subsequently revived after new maintainers added GOVERNANCE.md and increased bus factor from 2 to 4, illustrating that our indicators identify addressable risk factors.

5.2 Limitations and Threats to Validity

We discuss threats to validity following established guidelines [19]:

5.2.1 Internal Validity: Sustainability definition. Our binary sustainability classification (sustainable vs. non-sustainable) may oversimplify a continuous reality. Projects exist on a spectrum from thriving to declining to abandoned. We addressed this threat through:

- Sensitivity analysis using three alternative definitions (6-month, 12-month, 18-month activity thresholds)
- All definitions produced consistent results (Spearman correlations > 0.85 between alternative labels)
- Explicit archived status provides ground-truth for clearly non-sustainable projects

Missing data. 47% of projects lacked OpenSSF Scorecard data, potentially biasing our ecosystem-level analysis. We addressed this through:

- Multiple Imputation by Chained Equations (MICE) for missing values
- Complete-case sensitivity analysis showing consistent results
- Exclusion of OpenSSF scores from primary analysis, limiting their role to supplementary validation

Temporal confounding. Our cross-sectional design captures projects at different lifecycle stages. A project’s current governance may result from rather than cause its sustainability. Longitudinal studies are needed to establish causal direction.

5.2.2 External Validity: GitHub-only sample. Results are limited to GitHub projects and may not generalize to:

- GitLab or Bitbucket repositories
- Self-hosted Git repositories (corporate or academic)
- Non-software projects using GitHub for documentation or data
- Projects using alternative version control systems

Popularity bias. Our 500-star minimum threshold biases toward already-visible projects. Early-stage or niche projects may exhibit different sustainability patterns. Practitioners should exercise caution when applying our thresholds to projects below 500 stars.

Language and domain scope. We sampled only five programming languages (Python, TypeScript, JavaScript, Go, Java). Results may not generalize to:

- Systems programming (Rust, C, C++)
- Enterprise languages (C#, Scala)
- Scientific computing (R, Julia, MATLAB)
- Mobile development (Swift, Kotlin)

Temporal scope. Our snapshot represents December 2023–2024 conditions. OSS sustainability patterns may evolve with changing developer practices, funding models, and platform features.

Prevalence paradox. Our balanced sample (50% sustainable, 50% non-sustainable) optimizes model training but does not reflect the true population base rate. Industry reports suggest only 9% of OSS components remain actively maintained [17]. Consequently, the model’s precision would be

lower when applied to a random stream of GitHub repositories, as the 97.2% accuracy is calibrated against a 50/50 distribution rather than the 9:91 real-world ratio. Practitioners should interpret our thresholds as relative risk indicators rather than absolute probabilities.

Corporate backing confounder. Some sustainable projects in our sample may benefit from corporate sponsorship (e.g., projects maintained by employees of Google, Microsoft, or Facebook). Such corporate-backed projects have resources unavailable to purely volunteer-driven efforts. We did not control for corporate affiliation, so our “sustainability” findings may partially reflect “corporate funding” rather than governance practices alone. Future research should stratify by funding source.

5.2.3 Construct Validity: Causation vs. prediction. This is an observational study; all findings represent predictive associations, not causal relationships. We explicitly frame our contributions as predictors rather than interventions:

- We cannot claim that adding GOVERNANCE.md *causes* sustainability
- We can only say that governance presence *predicts* sustainability
- Confounding is likely: skilled maintainers may implement both governance AND sustain projects

Metric validity. Our operationalizations of bus factor (80% commit threshold) and contributor Gini may not perfectly capture the intended constructs:

- Bus factor counts commits, not influence or knowledge
- Some projects centralize commits through bots or single committers who merge PRs
- Gini coefficient treats all commits equally regardless of complexity

Label accuracy. Our sustainability labels depend on GitHub’s archived flag and commit activity, which may not capture:

- Projects in “completed” states (fully functional, no updates needed)
- Projects with activity in other channels (forums, Discord, mailing lists)
- Projects maintained through security patches only

5.2.4 Reliability: Replicability. All data collection scripts and analysis code are available in our replication package. We used deterministic random seeds for reproducibility of cross-validation splits.

Statistical robustness. We validated results through:

- FDR correction for multiple testing (Benjamini-Hochberg)
- Bootstrap confidence intervals (1000 resamples)
- Post-hoc power analysis confirming 99% power
- Cross-validation with 5 folds for predictive models

6 CONCLUSIONS

We analyzed 500 GitHub projects across governance, community, and ecosystem dimensions to identify factors associated with OSS sustainability. Our findings provide evidence-based guidance for practitioners assessing project health.

6.1 Summary of Findings

RQ1: Governance practices. Maintainer guidelines (MAINTAINERS.md, GOVERNANCE.md, CODEOWNERS) show the strongest governance association with sustainability (OR=4.22, $p < 0.0001$). PR templates also strongly predict sustainability (OR=3.17), while basic documentation (README, LICENSE) is universal and non-differentiating. Unexpectedly, issue templates show negative association, possibly reflecting reactive adoption by struggling projects.

RQ2: Community health. Bus factor of 4 distinguishes sustainable from non-sustainable projects, providing a concrete threshold for key-person risk assessment. Sustainable projects respond 3x faster to issues (0.54 vs. 1.78 days median). Higher contributor Gini in sustainable projects suggests that concentrated but distributed leadership outperforms shallow broad contributions.

RQ3: Ecosystem metrics. Stars threshold of approximately 12,000 provides a meaningful sustainability indicator (4.15x lift), but stars alone are insufficient. Forks and watchers show similar but weaker patterns. All ecosystem metrics exhibit diminishing returns above threshold values.

RQ4: Combined prediction. XGBoost achieves 97.2% accuracy (AUC=0.993), demonstrating that multi-dimensional assessment substantially outperforms single-factor evaluation. SHAP analysis reveals that stars and commit activity dominate predictions, but governance features provide incremental value.

6.2 Broader Implications

Our findings have implications beyond individual project assessment:

For the OSS ecosystem. The strong predictive power of governance documentation suggests that ecosystem-level sustainability could be improved through governance education and tooling. Platforms could highlight governance status in project discovery interfaces.

For funding organizations. The 97.2% prediction accuracy provides a foundation for evidence-based funding allocation. Organizations can prioritize projects with governance potential but low bus factor, where investment can have highest marginal impact.

For research. Our multi-dimensional framework and validated metrics provide a foundation for future sustainability research. The SHAP-based interpretation approach can be applied to other software engineering prediction tasks.

6.3 Practical Recommendations

Based on our findings, we provide concrete recommendations for different stakeholder groups:

For maintainers:

- Add MAINTAINERS.md or GOVERNANCE.md before seeking adoption
- Target bus factor of at least 4 through active contributor mentoring
- Prioritize issue response time under 1 day to signal active maintenance

- Use PR templates to standardize contribution workflows

For contributors:

- Check bus factor (≥ 4) before committing significant effort
- Evaluate issue response times (< 2 days indicates healthy project)
- Look for explicit governance documentation as quality signal
- Consider whether your contribution can improve project bus factor

For organizations:

- Use multi-factor checklist for dependency assessment: stars $> 12k$, bus factor ≥ 4 , governance docs present, response time < 2 days
- Don't rely solely on star counts for dependency decisions
- Consider governance maturity as leading indicator of sustainability
- Monitor dependency health through periodic reassessment

6.4 Future Work

We identify several directions for future research:

Causal validation. Longitudinal studies should examine whether implementing recommended practices (adding governance documentation, increasing bus factor) causally improves sustainability. Randomized controlled trials are ethically challenging, but natural experiments around governance policy changes could provide causal evidence.

Platform extension. Our study is limited to GitHub; extension to GitLab, Bitbucket, and self-hosted solutions would test generalizability. Different platform affordances (e.g., GitLab's built-in CI/CD) may affect sustainability patterns.

Tooling development. Our findings could be operationalized in automated sustainability scoring tools. Integration with package managers (npm, PyPI, Cargo) could help developers make informed dependency decisions at import time.

Early-stage projects. Our 500-star threshold excludes early-stage projects. Future work should investigate whether sustainability indicators differ for emerging projects and whether early governance adoption predicts later success.

REFERENCES

- [1] K. Crowston and J. Howison, "The social structure of free and open source software development," *First Monday*, vol. 10, no. 2, 2005.
- [2] I. Chengalur-Smith, A. Sidorova, and S. L. Daniel, "Sustainability of free/libre open source projects: A longitudinal study," *J. Assoc. Inf. Syst.*, vol. 11, no. 11, pp. 657–683, 2010.
- [3] J. Coelho and M. T. Valente, "Why modern open source projects fail," in *Proc. ESEC/FSE*, 2017, pp. 186–196.
- [4] J. Khondhu, A. Capiluppi, and K.-J. Stol, "Is it all lost? A study of inactive open source projects," in *Proc. OSS*, 2013, pp. 61–79.
- [5] C. M. Schweik and R. C. English, *Internet Success: A Study of Open-Source Software Commons*. Cambridge, MA: MIT Press, 2012.
- [6] S. O'Mahony and F. Ferraro, "The emergence of governance in an open source community," *Acad. Manage. J.*, vol. 50, no. 5, pp. 1079–1106, 2007.
- [7] Open Source Security Foundation, "OpenSSF Scorecard," 2024. [Online]. Available: <https://securityscorecards.dev/>
- [8] K. Fogel, *Producing Open Source Software: How to Run a Successful Free Software Project*. Sebastopol, CA: O'Reilly Media, 2005.

- [9] S. P. Goggins, K. Lombard, and M. Germonprez, "Open source community health: Analytical metrics and their corresponding narratives," in *Proc. SoHeal*, 2021, pp. 25–33.
- [10] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "A novel approach for estimating truck factors," in *Proc. ICPC*, 2016, pp. 1–10.
- [11] E. Constantinou and T. Mens, "Socio-technical evolution of the Ruby ecosystem in GitHub," in *Proc. SANER*, 2017, pp. 34–44.
- [12] C. Jergensen, A. Sarma, and P. Wagstrom, "The onion patch: Migration in open source ecosystems," in *Proc. ESEC/FSE*, 2011, pp. 70–80.
- [13] H. Borges, A. Hora, and M. T. Valente, "Understanding the factors that impact the popularity of GitHub repositories," in *Proc. ICSME*, 2016, pp. 334–344.
- [14] M. Valiev, B. Vasilescu, and J. Herbsleb, "Ecosystem-level determinants of sustained activity in open-source projects," in *Proc. FSE*, 2018, pp. 401–411.
- [15] D. He *et al.*, "Six million (suspected) fake stars in GitHub: A growing spiral of popularity contests, scams, and malware," *arXiv:2412.13459*, Dec. 2024.
- [16] K.-J. Stol and B. Fitzgerald, "The ABC of software engineering research," *ACM Trans. Softw. Eng. Methodol.*, vol. 27, no. 3, pp. 11:1–11:51, 2018.
- [17] Black Duck, "Open source security and risk analysis (OSSRA) report 2024," Black Duck Software, San Jose, CA, Tech. Rep., 2024.
- [18] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Stat. Soc. Ser. B*, vol. 57, no. 1, pp. 289–300, 1995.
- [19] A. Mockus, R. T. Fielding, and J. D. Herbsleb, "Two case studies of open source software development: Apache and Mozilla," *ACM Trans. Softw. Eng. Methodol.*, vol. 11, no. 3, pp. 309–346, 2002.
- [20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [21] M. Zhou and A. Mockus, "What make long term contributors: Willingness and opportunity in OSS community," in *Proc. ICSE*, 2012, pp. 518–528.
- [22] T. Menzies and T. Zimmermann, "Software analytics: So what?" *IEEE Software*, vol. 30, no. 4, pp. 31–37, 2013.